

第5章 Montevina 的 MCH 和 ICH

本章以 Montevina 平台为例，说明在 x86 处理器系统中，PCIe 体系结构的实现机制。Montevina 平台是 Intel 提供的一个笔记本平台。在这个平台中，含有一个 Mobile 芯片组、Mobile 处理器和无线网卡。其中 Mobile 芯片组包括代号为“Contiga”的 GMCH（Graphics and Memory Controller Hub）和 ICH9M 系列的 ICH；Mobile 处理器使用代号为“Penryn”的第二代 Intel Core2 Duo；无线网卡的代号为“Shirley Peak”（支持 WiFi）或者“Echo Peak”（同时支持 WiFi 和 WiMax）。Montevina 平台的拓扑结构如图 5-1 所示。

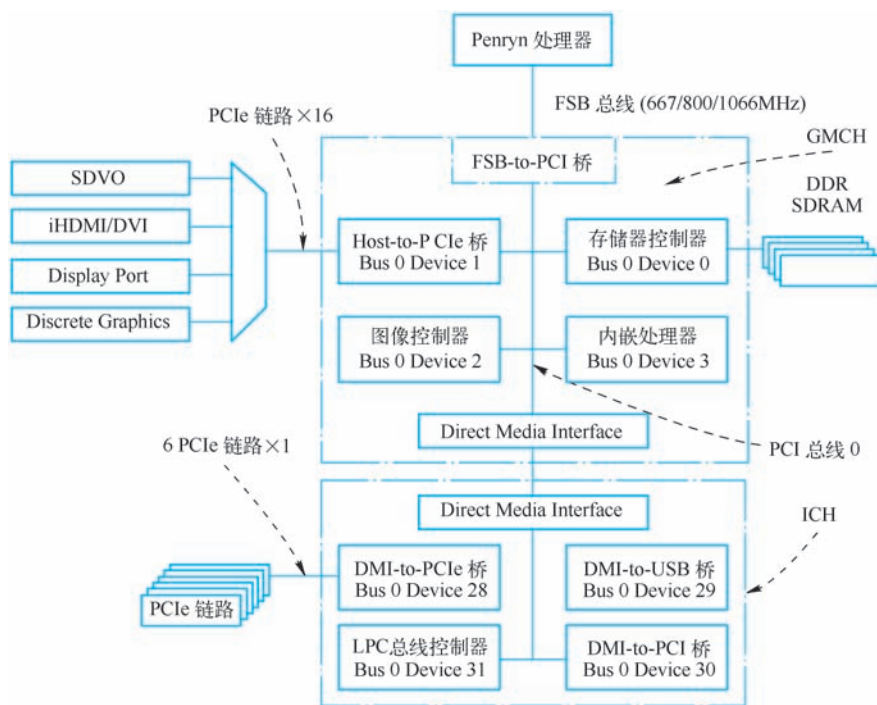


图 5-1 Montevina 平台的拓扑结构

Montevina 平台使用一个虚拟的 FSB-to-PCI 桥^①将 FSB 总线与外部设备分离，这个虚拟 PCI 桥的上方连接 FSB 总线，之下连接 PCI 总线 0。但是从物理信号的角度来看，MCH 中的 PCI 总线 0 是 FSB 总线的延伸，因为该 PCI 总线 0 依然使用 FSB 总线的信号，只是连接到这条总线上的设备相当于虚拟 PCI 设备。在 GMCH 中，并没有提及这个 FSB-to-PCI 桥，但是在芯片设计中，存在这个桥片的概念。

从系统软件的角度来看，在 PCI 总线 0 上挂接的设备都含有 PCI 配置寄存器，系统软件将这些设备看作 PCI 设备，并可以访问这些设备的 PCI 配置空间。在 Montevina 平台的

① 在 Montevina 平台的数据手册中并没有提及这个 FSB-to-PCI 桥。

GMCH 和 ICH 中，所有的外部设备，如存储器控制器，图形控制器等都是虚拟 PCI 设备，都具有独立的 PCI 配置空间。GMCH 和 ICH 之间使用 DMI（Direct Media Interface）接口相连，但是 DMI 接口仅仅是链路级别的连接，并不产生新的 PCI 总线号，ICH 的 DMI-to-USB 桥和 DMI-to-PCIe 桥也都属于 PCI 总线 0 上的设备。

在 x86 处理器中，MCH 包含的虚拟 PCI 设备优先级较高，而 ICH 包含的虚拟 PCI 设备优先级较低。当 CPU 发起一个 PCI 数据请求时，MCH 的 PCI 设备将首先在 PCI 总线 0 上进行正向译码。如果当前 PCI 数据请求所使用的地址没有在 MCH 的 PCI 设备命中时，DMI 接口部件将使用负向译码方式被动地接收这个数据请求，然后通过 DMI 总线将这个数据请求转发到 ICH 中。

因此在 x86 处理器中，MCH^①集成了一些对带宽要求较高的虚拟 PCI 设备，如 DDR 控制器、显卡等。而在 ICH 中集成了一些低速 PCIe 端口，和一些速度相对较低的外部设备，如 PCI-to-USB 桥、LPC 总线控制器等。

MCH 和 ICH 包含一些内置的 PCI 设备，这些设备都具有 PCI 配置空间，x86 处理器可以使用 PCI 配置周期访问这些 PCI 配置空间。在 MCH 和 ICH 中，PCI 总线 0 是 FSB 总线的延伸，所以处理器访问这些设备时并不使用 PCI 总线规定的信号，如 FRAME#、TRDY#、IRDY#和 IDSEL 信号。在 MCH 和 ICH 中，有些 PCI 设备并不是传统意义上的外部设备，而仅是虚拟 PCI 设备，即使用 PCI 总线的管理方法统一在一起的设备。

x86 处理器使用这些虚拟 PCI 外设的优点是可以将所有外部设备都用 PCI 总线统一起来，这些设备使用的寄存器都可以保存在 PCI 设备的配置空间中，但是使用这种方法在某种程度上容易混淆一些概念，尤其是有关地址空间的概念。例如在处理器体系结构的典型定义中，DDR SDRAM 空间属于存储器域，与其相关的 DDR SDRAM 控制器也应该属于存储器域，但是在 x86 处理器中存储器控制器属于 PCI 总线域。

5.1 PCI 总线 0 的 Device 0 设备

PCI 总线 0 上存储器控制器（Device 0）是一个比较特殊的 PCI 设备，这个设备除了需要管理 DDR SDRAM 之外，还管理整个存储器域的地址空间，包括 PCI 总线域地址空间。在 x86 处理器系统中，该设备是管理存储器域空间的重要设备，其中含有许多与存储器空间相关的寄存器。

这些寄存器对于系统程序员理解 x86 处理器的存储器拓扑结构非常重要，对底层编程有兴趣的系统程序员需要掌握这些寄存器。但是在 x86 处理器系统中，由于 BIOS 的存在，绝大多数系统程序员并没有机会实际使用这些寄存器。

从底层开发的角度上看，x86 处理器系统并不如 PowerPC、MIPS 和 ARM 处理器透明。x86 处理器首先使用 BIOS 屏蔽了处理器的硬件实现细节，其次在处理器内核中使用了 Microcode^②进一步屏蔽了 CPU 的实现细节。这使得底层程序员在没有得到充分的资源时，几乎无法开发 x86 处理器的底层代码。

① 从体系结构的角度上看，MCH 和 ICH 仅仅是一个称呼，实际上并不重要。

② 许多处理器和外部设备都使用了 Microcode 屏蔽 CPU 的实现细节，如 Alpha 处理器的 PAL。

但是不可否认的是 x86 处理器底层开发的复杂程度超过 PowerPC、MIPS 和 ARM 处理器，因为 x86 处理器系统作为通用 CPU 需要与各类操作系统兼容，而向前兼容对于任何一种处理器都是一个巨大的包袱。x86 处理器系统使用 BIOS 和 Microcode 屏蔽硬件细节基于许多深层次的考虑，包括技术和商业上的考虑，这种做法在 PC 领域取得了巨大的成功。

从传统外部设备的角度上看，PCI 总线 0 的 Device 0 并不是一个设备，仅存放与处理器系统密切相关的一组参数。而除了 x86 处理器之外，几乎所有处理器都使用存储器映射寻址的寄存器保存这些参数。

x86 处理器需要考虑向前兼容，因此存在许多独特的设计。这些独特的设计极易使一些初学者混淆计算机体系结构中的一些基本概念。从这个角度来看，x86 处理器并不适合教学，但这并不影响 x86 处理器在 PC 领域的地位。值得注意的是，在 x86 处理器中，PCI 总线 0 的 Device 0 的存在并不完全是为了向前兼容，而是 Intel 使用 PCI 总线概念统一所有外部设备的方法。

在 Montevina 平台中，系统软件使用 Type 00h 配置请求访问存储器控制器，该存储器控制器除了具有一个标准 PCI Agent 设备的 64B 的配置空间之外，还使用了 PCI 设备的扩展配置空间，其包含的主要寄存器如表 5-1 所示。

表 5-1 Device 0 的基本配置空间

寄存器名	简 写	缺 省 值	说 明
Vendor Identification	VID	0x8086	Intel 公司使用的 VID
Device Identification	DID	0x2A40	Contiga 使用的 DID
PCI Command	PCICMD	0x0006	支持存储器空间，可作为主设备
PCI Status	PCISTS	0x0090	支持背靠背传送和 Capability 寄存器
Revision Identification	RID	0x00	版本号为 0
Class Code	CC	0x060000	表示该设备为 Host Bridge
Master Latency Timer	MLT	0x00	PCIe 设备不再使用该寄存器
Header Type	HDR	0x00	表示为 PCI 单功能设备
Subsystem Vendor Identification	SVID	0x0000	未使用
Subsystem Identification	SID	0x0000	未使用
Capability Pointer	CAPPTR	0xE0	第一个 Capability 寄存器地址为 0xE0

Device 0 使用的基本配置空间与其他 PCI 设备兼容。这里值得注意的是 Device 0 在 PCIe 体系结构中，被认为是 HOST 主桥。而 Device 0 使用的 PCI 扩展配置空间也被称为 RCRB，RCRB 的主要作用是描述当前处理器的存储器地址拓扑结构，包括主存储器地址和 PCI 总线地址。其简写和复位值如表 5-2 所示。

表 5-2 Device 0 的扩展 PCI 配置空间

寄存器名	简 写	复 位 值
Egress Port Base Address	EPBAR	0x0000-0000-0000-0000
(G) MCH Memory Mapped Register Range Base	MCHBAR	0x0000-0000-0000-0000

(续)

寄存器名	简 写	复 位 值
(G) MCH Graphics Control Register	GGC	0x0030
Device Enable	DEVEN	0x000043DB
PCI Express Register Range Base Address	PCIEXBAR	0x0000-0000-E000-0000
MCH-ICH Serial Interconnect Ingress Root Complex	DMIBAR	0x0000-0000-0000-0000
Programmable Attribute Map 0	PAM0	0x00
Programmable Attribute Map 1	PAM1	0x00
Programmable Attribute Map 2	PAM2	0x00
Programmable Attribute Map 3	PAM3	0x00
Programmable Attribute Map 4	PAM4	0x00
Programmable Attribute Map 5	PAM5	0x00
Programmable Attribute Map 6	PAM6	0x00
Legacy Access Control	LAC	0x00
Remap Base Address Register	REMAPBASE	0x03FF
Remap Limit Address Register	REMAPLIMIT	0x0000
System Management RAM Control	SMRAM	0x02
Extended System Management RAM Control	ESMRAMC	0x38
Top of Memory	TOM	0x0001
Top of Upper Usable DRAM	TOUUD	0x0000
Top of Low Used DRAM Register	TOLUD	0x0010
Error Status	ERRSTS	0x0000
Error Command	ERRCMD	0x0000
Scratchpad Data	SKPD	0x0000-0000
Capability Identifier	CAPID0	0x0000-0000-0000-010A-0009

系统软件首先检查 Capability Identifier 寄存器，该寄存器的地址偏移为 0xE0，即 CAPPTR 寄存器指向的地址为 0xE0。该 Capability 结构使用的 PCI Express Extended Capability ID 字段（该字段在 CAPID0 寄存器中）为 0x0A，因此表 5-2 中的寄存器组为 RCRB Capability 结构，有关 Capability 结构的组成结构见第 4.3 节。

在 x86 处理器系统中，RCRB 存放一些与处理器系统相关的寄存器。而在许多处理器中，如在 PowerPC 处理器中并不含有 RCRB。在 PowerPC 处理器中，与处理器系统相关的寄存器都存放在以 BASE_ADDR 为起始地址的 1MB 连续的物理地址空间中，PowerPC 处理器系统使用存储器映射寻址方式访问这些寄存器。

在 x86 处理器系统中，使用 PCI 总线管理所有外部设备，这些“与处理器系统相关的寄存器”被保存在 RCRB 中，处理器使用 PCI 总线配置周期访问这些寄存器。实际上在 RCRB 中包含的寄存器与 PCIe 体系结构并没有直接关系，这些寄存器应该属于存储器域的地址区域。x86 处理器的这种做法并非完全合理，在某种程度上容易使初学者混淆存储器域与 PCI 总线域的区别。RCRB 主要寄存器的含义如下所示。

5.1.1 EPBAR 寄存器

EPBAR 寄存器的大小为 8B，指向一个 4 KB 大小的存储器区域。处理器使用存储器映像寻址访问这段存储器区域，并通过这段存储器区域访问 RCRB 的扩展配置空间，在表 5-2 中存放的仅是 RCRB 的部分扩展配置空间。

这段存储器区域描述 RC 的 Egress 端口属性，包括 RC 使用的 VC0 和 VC1 两个虚通路的具体信息。当 EPBAR 寄存器的“EPBAR Enable”位为 1 时，这段空间有效。这段寄存器区域被称为“Egress Port RCRB”空间，系统软件可以使用这段空间定义的寄存器，完成对 VC1 和 VC0 通路的设置，包括端口仲裁、VC 仲裁等一系列的内容。

5.1.2 MCHBAR 寄存器

MCHBAR 寄存器的大小为 8B，指向一个 16 KB 大小的存储器区域。处理器使用存储器映像寻址访问这段存储器区域。这段存储器区域描述 GMCH 内部使用的一些寄存器。当 MCHBAR 寄存器的“MCHBAR Enable”位为 1 时，这段存储器区域有效。

在这段区域中含有多组寄存器。

- Device 0 Memory Mapped I/O 寄存器组。包括 MEREMAPBAR、GFXREMAPBAR、VCOREMAPBAR、VC1REMAPBAR 和 PAVPC 寄存器。
- DRAM Channel Control 寄存器。该寄存器用来设置 x86 处理器系统中的 DRAM 通路。在 Montevina 平台中含有两个 DRAM 通路。这两个 DRAM 通路可以独立使用，也可以设置成 Interleaved 模式。
- MCHBAR Clock Control 寄存器组。由 CLKCFG 和 SSKPD 寄存器组成，其中 CLKCFG 寄存器可以设置 DDR 使用的频率和 FSB 总线的频率；而 SSKPD 寄存器供 BIOS 或者 Graphic 驱动使用，用来保存一些中间结果。
- Device 0 MCHBAR ACPI Power Management Controls 寄存器组。该组寄存器与 x86 处理器系统使用的 ACPI 有关。
- Device 0 MCHBAR Thermal Management Controls 和 MCHBAR Render Thermal Throttling 寄存器组。该组寄存器与 Montevina 平台中的温度传感器相关。
- Device 0 MCHBAR DRAM Controls 寄存器组。这组寄存器对 Montevina 平台中的两个 DRAM 通路进行细粒度的控制，包括这两个 DRAM 通路中使用的 Timing、延时和各种模式选择。该寄存器组对于需要设置 DRAM 属性的 BIOS 工程师非常重要。

5.1.3 其他寄存器

在 Device 0 中还包含以下寄存器。

- GGC 寄存器。在 x86 处理器系统中，显卡可以借用一部分存储器域的地址空间，该寄存器描述这段被借用的存储器地址空间。
- DEVEN 寄存器。该寄存器用来使能/禁止在 MCH 中的虚拟 PCI 设备，如显卡控制器和其他虚拟设备。
- PCIEXBAR 寄存器。该寄存器的大小为 8B，指向一个 256 MB 大小的存储器区域。PCIe 总线可以使用 ECAM 方式访问 PCI 设备的扩展配置空间，PCIEXBAR 寄存器存

放 PCI 配置空间的基地址，有关 ECAM 机制的详细信息见第 5.3.2 节。

- DMIBAR 寄存器。该寄存器的大小为 8B，指向一个 4 KB 大小的存储器区域，处理器使用存储器映像寻址访问这段存储器区域。该寄存器描述 RC 中的 DMI 接口。
- PAM0~PAM6 寄存器描述 Shadow BIOS 的属性。
- REMAPBASE 寄存器和 REMAPLIMIT 寄存器支持 x86 处理器的 Reclaim 机制，第 5.2 节将详细介绍该机制。
- SMRAM 寄存器和 ESMRAMC 寄存器控制处理器对 SMRAM 的访问。SMRAM 与 x86 处理器的 SMM 机制相关，本书对此不做介绍。
- TOM、TOUUD 和 TOLUD 寄存器与处理器系统的主存储器管理相关，分别描述存储器的物理大小和存储器“低于 4 GB”和“高于 4 GB”地址空间的使用方法。第 5.2.3 节将详细介绍这几个寄存器。

5.2 Montevina 平台的存储器空间的组成结构

由上文所述，在 Montevina 平台中包含一个 Mobile 处理器、MCH、ICH 和一个无线网卡适配器组成。在 MCH 和 ICH 中具有许多组成部件，本节仅介绍 MCH 和 ICH 中与 PCI 总线直接相关的部分内容。

Montevina 平台使用的地址空间由存储器域地址空间和 PCI 总线域地址空间组成。在 Intel 的 x86 处理器系统中，包括 Montevina 平台，所有的外部设备都通过 PCI 总线进行管理。x86 处理器平台使用这种方法便于对外部设备统一管理，但是这种方法也带来了一些弊端。因为使用这种方法时，PCI 总线域空间与存储器域空间的边界划分并不明晰。

Montivina 平台除了具有存储器域、PCI 总线域之外，还存在一个 DRAM 域。所谓 DRAM 域是指 DRAM 控制器所能访问的地址空间，即从 DRAM 控制器的角度来看，DRAM 空间的拓扑结构。DRAM 域中包含的地址空间，通俗地讲是指主存储器地址空间，即 DRAM 控制器能够访问的地址空间。

在 Montevina 平台中，DRAM 域地址空间并不能与存储器域地址空间完全对应。当处理器系统支持的内存超过 4 GB 时，DRAM 域的部分空间需要使用 Reclaim 机制才能访问，此外在 DRAM 域空间中，有些地址并不能被处理器访问。比如显卡控制器借用了一部分 DRAM 空间，这部分空间可以被显卡控制器访问，但是不能被 CPU 访问。x86 处理器由于考虑向前兼容，一个原本完整的 DRAM 域被划分得支离破碎。在 Intel 的 x86 处理器中，许多“不合理”都是因为“向前兼容”导致的。

在 x86 处理器系统中，存储器域由 CPU 能够访问的地址空间组成，包括 DRAM 域地址空间的一部分，一些使用存储器映像寻址的寄存器^①和 PCI 总线域地址空间在存储器域中的映像。

而 DRAM 域由 DRAM 控制器所能寻址的空间组成。如图 5-2 所示，在 Montevina 平台中，存储器域与 DRAM 域的所包含的部分空间，其地址相等，比如 Legacy Address Range、

① x86 处理器也将使用存储器映像寻址的寄存器称为 MMIO (Memory Mapped I/O)。

TSEG (Top of Memory Segment) 和其他一些 DRAM 空间。这里的地址相等指在存储器域和 DRAM 域中的地址相同，但是这两个地址的含义并不相同。

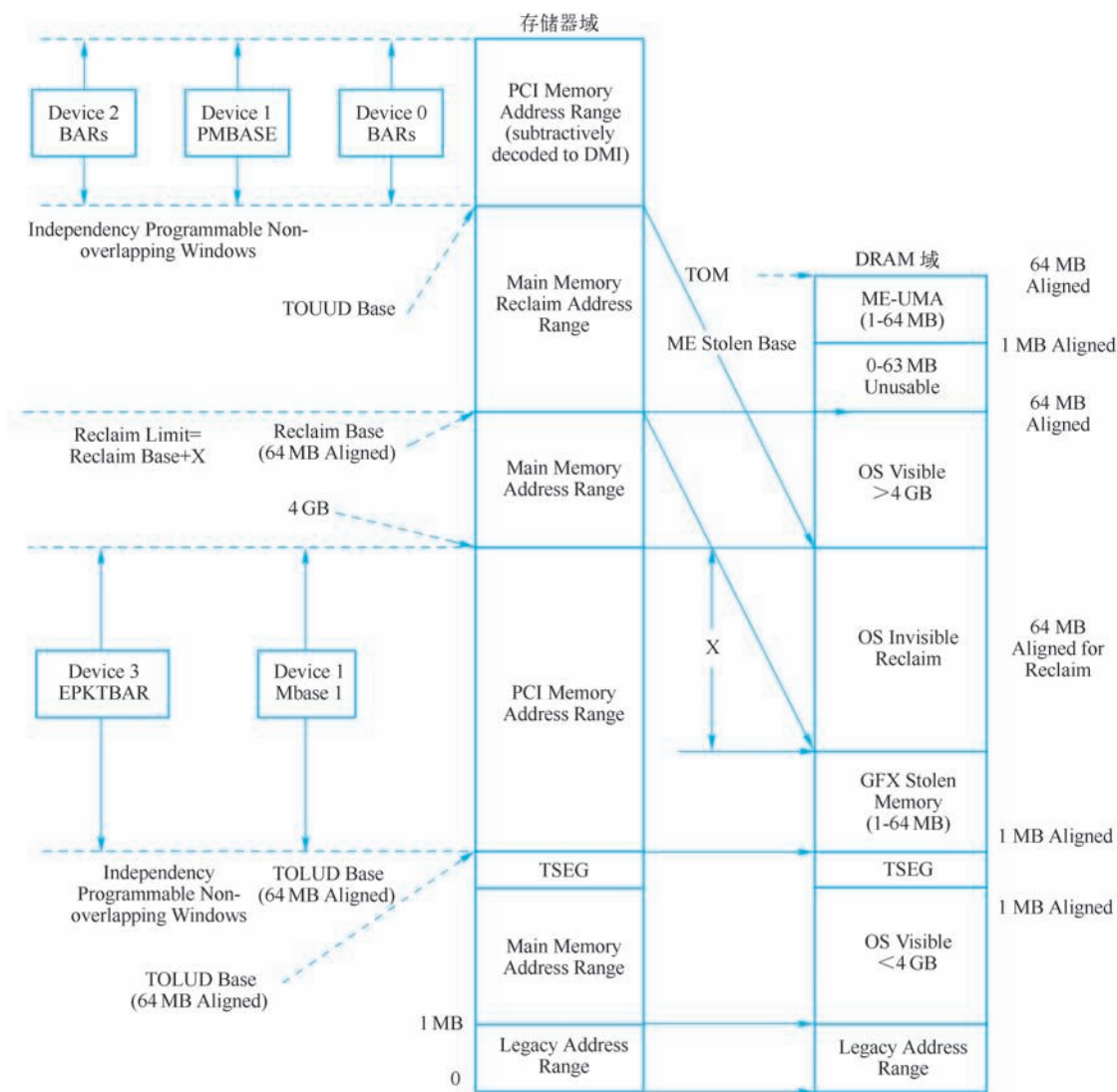


图 5-2 Montevina 平台的 CPU 域与 DRAM 域

在一个多核处理器系统中，不同的 CPU 所能访问的地址空间也不一定相同，其中每一个 CPU 都对应一个存储器域。在这些存储器域中，有些空间是所有 CPU 共享的，有些空间是某个 CPU 的私有空间。在多核处理器中，存储器域和 DRAM 域地址空间的划分更为复杂，本书对此不做进一步说明。

如图 5-2 所示，x86 处理器将 PCI 总线域和存储器域进行混合编址。但是在图 5-2 中的 PCI 总线地址仅是在存储器域中的地址，即 PCI 总线地址在存储器域地址空间的映像。值得注意的是，这个 PCI 总线地址和 PCI 总线域的地址没有直接联系，当处理器访问 PCI 设备时，首先使用在存储器域的 PCI 总线地址，RC 会将存储器域的地址转换为 PCI 总线域的地

址，并使用 PCIe 总线事务访问相应的设备。

x86 处理器和 PowerPC 处理器进行存储器域到 PCI 总线域的映射方法不同。PowerPC 处理器使用 Inbound/Outbound 窗口显式地分离存储器域与 PCI 总线域。而 x86 处理器内部并没有设置这类寄存器显式分离这些域空间。但是 x86 处理器仍然区分存储器域和 PCI 总线域，虽然 PCI 设备使用的地址在存储器域和 PCI 总线域中相同。

x86 处理器采用的这种 PCI 总线域与其他处理器有较大的不同，x86 处理器采用这种设计方法可能考虑了向前兼容。采用这种结构，存储器域和 PCI 总线域之间的界限并不明晰，但是有利于外部设备的统一管理。

5.2.1 Legacy 地址空间

Legacy 地址空间在存储器域和 DRAM 域中都有映射，且地址相等，这段空间是 x86 处理器所固有的一段大小为 1 MB 的内存空间，它伴随着 x86 处理器的整个发展历程。Legacy 地址空间的大小为 1 MB，其详细说明如下。

- 0x0000-0000 ~ 0x0009-FFFF，DOS 使用的 640 KB 大小的基本内存空间。目前开发 BIOS 的工程师仍然在 DOS 下工作。
- 0x000A-0000 ~ 0x000B-FFFF，Legacy Video 空间，大小为 128 KB。这段空间作为 VGA 设备的 Frame Buffer。目前使用的高端显卡，处于 VGA 模式时，也使用这段空间作为 Frame Buffer。
- 0x000A-0000 ~ 0x000B-FFFF，Compatible SMRAM 空间，大小为 128 KB。
- 0x000B-0000 ~ 0x000B-7FFF，MDA (Monochrome Display Adapter) 空间，大小为 32 KB。MDA 是一种非常老的单色显示设备。
- 0x000C-0000 ~ 0x000D-FFFF，ISA 扩展区域，大小为 128 KB。
- 0x000E-0000 ~ 0x000E-FFFF，BIOS 扩展区域，大小为 128 KB。
- 0x000F-0000 ~ 0x000F-FFFF，系统 BIOS 区域，大小为 128 KB。

5.2.2 DRAM 域

在处理器系统中，可能含有多个 DRAM 控制器，可以管理多个 DRAM 空间，这些空间可以统一编址，也可以独立编址，从而组成一个或者多个 DRAM 域。DRAM 域的大小由一个处理器系统中具有的物理内存大小决定，在绝大多数处理器系统中，DRAM 域的物理地址空间是连续的，其最低地址为 0x0。DRAM 域地址空间的设置与处理器系统使用的 DRAM 控制器相关。在 PowerPC 处理器中，DRAM 域的最低物理地址是可变的，而且 DRAM 域的地址空间也可以不连续。

而在多数 x86 处理器中，DRAM 域的物理地址，其基地址不可改写，值为 0x0。多数 BIOS 都将 DRAM 域的地址空间设置为一段连续的地址空间，但是 x86 处理器也支持不连续的 DRAM 区域。x86 处理器系统的 DRAM 控制器远比 Power 处理器复杂，只是多数系统软件程序员并没有关心这些细节。

在 Montevina 平台中，DRAM 域空间的大小，即 x86 处理器使用的主存储器大小，保存在 TOM (Top of Memory) 寄存器中，该寄存器在 MCH 的存储器控制器中，即 PCI 设备 0 的配置空间中。在 DRAM 域中除了 Legacy Address 空间之外，还包括以下几种空间。

1. GFX (显卡控制器) 借用的空间

这段空间的大小为 1 MB~64 MB, 由 GFX 管理, CPU 不能访问这段空间。这段空间的基地址保存在 GBSM (Graphics Base of Stolen Memory) 寄存器中, 其值为 TOLUD (Top of Low Usable DRAM) 寄存器的值。

TOLUD 和 GBSM 寄存器在存储器控制器中, 其中 TOLUD 寄存器保存 x86 处理器系统低端内存 (小于 4 GB) 的大小。TOLUD 寄存器的值由 BIOS 根据处理器系统的配置决定, 该寄存器的详细描述见下文。

2. TSEG 空间

TSEG 空间, 这段空间的大小为 1 MB~8 MB。这段空间仅在 CPU 进入 SMM 模式时, 才能够被 CPU 访问。这段空间的基地址保存在 TSEGMB (TESG Memory Base) 寄存器中, 其值为 TOLUD 寄存器减去 GFX 借用空间和 TSEG 空间的大小。TSEGMB 寄存器也保存在存储器控制器中。在 x86 处理器中, 还有一段进入 SMM 模式才能访问的地址空间, HSEG 空间。这段空间被处理器映射到高端地址, HSEG 和 TSEG 空间的区别在于 TSEG 空间是可 Cache 的, 而 HSEG 空间不可 Cache。

3. 小于 4 GB 的操作系统可用内存空间

在 Montevina 平台中, 如果主存储器的大小超过 4 GB, 那么这段物理空间将被分为三段供操作系统使用, 分别是小于 4 GB 的空间、大于 4 GB 的空间和 Reclaim 空间。

如图 5-2 所示, 这几段空间在存储器域中的地址并不连续。但是在 DRAM 域中, 这些地址空间是连续, 并从地址 0 到 TOM。而在存储器域中, 小于 4 GB 的内存空间从 1 MB 开始到 TOLUD 结束。

4. OS Invisible Reclaim 空间

对于 DRAM 控制器而言, 这段空间是存在的, 不过因为这段地址和存储器域中的 PCI 总线地址空间重合, 因此 CPU 不能直接访问这段物理内存。当处理器使能 Reclaim 机制 (也称为 Remapping 机制) 后, CPU 才能访问这段地址空间。

在一个 x86 处理器系统中, 当实际的物理内存大于 TOLUD 加上 GFX 借用空间的大小时, DRAM 域中才含有这段空间, 此时 CPU 才有必要启动 Reclaim 机制, 将这段内存地址空间映射到存储器域中。当然 CPU 也可以不启动 Reclaim 机制, 放弃对这段内存空间的使用。

5. 大于 4 GB 的操作系统可用空间

这段空间在存储器域和 DRAM 域中的地址相等, 只有实际的物理内存大于 4 GB 时, 存储器域和 DRAM 域中才有这段空间。

6. ME (Manageability Engine) -UMA 借用的空间

其大小为 1~64 MB。其基地址为 TOM 寄存器中的值减去 ME-UMA 借用空间的大小。由于 TOM 寄存器中的值需要 64 MB 对界, 因此有时在其下会出现一个 0~63 MB 大小的空洞。这段空洞是被浪费的空间, 除了 DRAM 控制器可以访问这段空间外, 在处理器系统中的其他部件均不能访问这段空间。

5.2.3 存储器域

在 x86 处理器系统中, 除了具有 DRAM 域地址空间外, 还具有 PCI 地址空间。CPU 访问这些地址空间时, 需要进行地址转换, CPU 访问 DRAM 域时, 需要进行存储器域地址空

间到 DRAM 域地址空间的转换；CPU 访问 PCI 总线域时，需要进行存储器域地址空间到 PCI 总线域地址空间的转换。在 x86 处理器中，大多数 DRAM 域中的地址与存储器域中的地址一一对应而且相等，而存储器域的 PCI 地址与 PCI 总线域的地址也一一对应而且相等。

在 x86 处理器中，并没有提及地址转换部件，但是这个地址转换部件的概念是存在于芯片设计中的。CPU 访问 DRAM 或者 PCI 设备时，首先需要访问存储器域的地址，这些发送到存储器域的数据请求首先通过地址转换部件，并由地址转换部件决定这些数据请求是发向 PCI 总线域、DRAM 域还是其他域空间。在 x86 处理器中，这些地址域的关系如图 5-3 所示。

在 Montevina 平台中，存储器域中的 Legacy Address 空间的地址与 DRAM 域空间的地址一一对应，而且地址相等。当 CPU 访问这段空间时，地址转换部件直接将发向存储器域的数据请求转发到 DRAM 域。而 CPU 访问其他段空间时，需要分别进行处理。

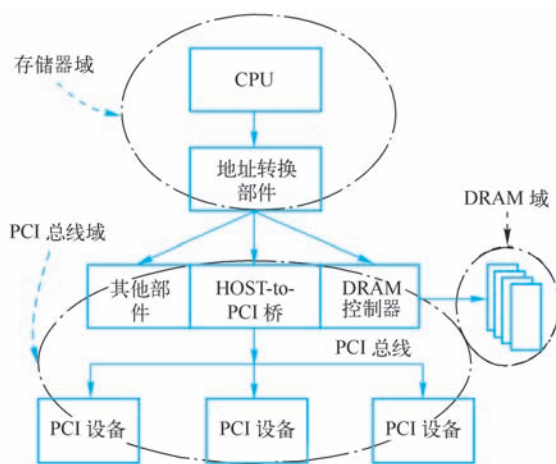


图 5-3 存储器域与 PCI/DRAM 域

1. 1 MB~TOLUD 空间

这段空间由三部分组成，GFX Stolen Memory 空间、TSEG 和 Main Memory Address 空间。当 CPU 访问 GFX Stolen Memory 空间时，地址转换部件将拒绝这次访问，因为 GFX Stolen Memory 空间是 GFX 控制器的私有空间，CPU 不能对这段空间进行操作。

TSEG 空间只能在 CPU 处于 SMM 状态时才能访问，否则处理器内部的地址转换部件也将拒绝这次访问。

Main Memory Address 空间与 DRAM 域中小于 4 GB 的操作系统可用空间一一对应且地址相等，当 CPU 访问这段地址空间时，地址转换部件直接将这次访问转发到 DRAM 域，并由存储器控制器访问这段空间。这段空间也是存储器域中第 1 段“主存储器”空间。

2. TOLUD~4 GB 空间

这段空间是存储器域中的 PCI 总线地址空间，当 CPU 访问这段地址空间时，地址转换部件将这次访问转发给 HOST 主桥或者 RC，之后由 HOST 主桥或者 RC 将 CPU 的这次数据访问转化为 PCI 总线的总线事务，之后再发送到对应的 PCI 设备。在 x86 处理器中，存储器域的 PCI 总线地址空间与 PCI 总线域的地址空间一一对应，其值完全相等。一些采用存储器映像寻址的寄存器也存放在这段空间中，如 APIC 中断控制器使用的地址空间。

3. 4 GB~TOUUD 空间

这段空间由 Main Memory Address 空间和 Main Memory Reclaim Address 空间组成。如图 5-2 所示，在一个 x86 处理器系统中，如果实际的物理内存空间大于 4 GB 时，存储器域中将含有第 2 段 Main Memory Address 空间。

这段空间与 DRAM 域中大于 4 GB 的操作系统可用空间一一对应，而且地址相等，CPU 访问这段地址空间时，地址转换部件直接将这次访问转发到 DRAM 域，这段空间的上界为

TOM 减去 ME (Manageability Engine) 借用的空间。ME 借用的地址空间不能被 CPU 访问, 本节对这段空间不做进一步介绍。

4. Reclaim 空间

如果 CPU 使能了 Reclaim 机制, 则存储器域具有这段空间。在 x86 处理器中存储器域与 DRAM 域基本上——对应而且地址相等, 但是 TOLUD~4 GB 这段空间例外。存储器域将这段空间分配为 PCI 总线地址空间, 因此 CPU 不能直接用“相同的地址”访问 DRAM 这段空间。只有 x86 处理器使用 Reclaim 机制, 将 DRAM 域的“OS Invisible Reclaim 空间”映射到存储器域的“Reclaim Base~TOUUD”这段空间后, 处理器才能访问这段空间。

这段空间也是 x86 处理器的第 3 段存储器空间。值得注意的是这段存储器空间在存储器域中的地址与 DRAM 域中的地址并不相等。

当 Reclaim 机制使能后, Reclaim Base 的值为 TOM 减去 ME 借用的空间, 这段空间的大小等于 4 GB 减去 TOLUD (64 MB 对界), 然后再减去 GFX 借用的空间大小。此时 TOUUD 的值等于 Reclaim Base 加上 Reclaim 空间的大小。

5. TOUUD~64 GB 空间

在 x86 处理器系统中, PCI 总线需要的空间大于 TOLUD~4 GB 这段区域时, 将使用这段空间映射剩余的 PCI 总线地址空间, 这段空间的使用方法与 BIOS 的设置有关。在 Montevina 平台中, CPU 使用的物理地址为 36 位, 因此存储器域的最大物理地址空间为 64 GB。

5.3 存储器域的 PCI 总线地址空间

由图 5-2 可知, Montevina 平台中存储器域的 PCI 总线地址空间分为 TOLUD~4 GB 和 TOUUD~64 GB 这两段空间。这两段空间都可以映射 PCI 设备使用的空间, x86 处理器为了实现向前兼容, 并没有取消 TOLUD~4 GB 这段空间, 而这段空间的存在将存储器域的 DRAM 空间一分为二。

5.3.1 PCI 设备使用的地址空间

TOLUD~4 GB 这段 PCI 总线地址空间主要映射和 ICH 相连的 PCI 设备地址空间, 此外还包括 EPBAR (Egress Port Base Address) 指向的空间, 以及 MCHBAR 和 DMIBAR 指向的空间等。除了 PCI 总线地址空间外, 这段空间还包括 High BIOS、APIC (Advanced Programming Interrupt Controller) 和 FSB Interface 地址空间, 其详细描述如图 5-4 所示。

其中 APIC (Advanced Programmable Interrupt Controller) 包括 I/O APIC 和 Local APIC 占用 0xFEC0-0000~0xFECF-FFFF 这段地址空间。APIC 是 x86 处理器使用的中断控制器, 负责管理外部和 CPU 之间的中断请求。而 HSEG 空间占用 0xFEDA-0000~0xFEDB-FFFF 这段地址空间, 这段空间在 CPU 内核处于 SMM 状态时, 才能访问。

FSB Interface 存储器空间与 MSI 中断机制相关, PCIe 设备向这段存储器空间进行写操作时, MCH 将这个写操作转换为 FSB 总线的 Interrupt Message 总线事务。

值得注意的是, 在 x86 处理器中 Local APIC 使用的寄存器在 0xFEE0-0000~0xFEE0-03F0 区域之间, 在这段区域中, 有一些 Reserved 寄存器, 如 0xFEE0-0000~0xFEE0-0010, 系统软件不能操作这些寄存器。Intel 并没有公开这些寄存器的具体含义, 但是从原理上推

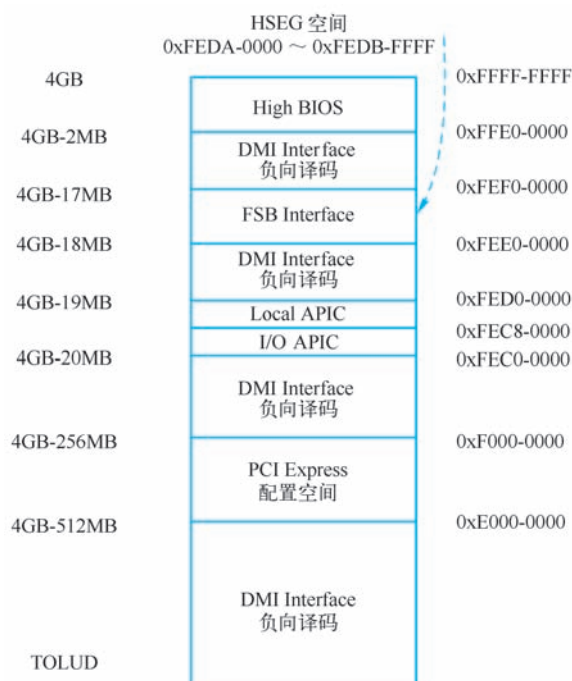


图 5-4 TOLUD~4 GB PCI 总线地址空间

断，这段寄存器所使用的地址正好是 PCIe 设备使用 MSI-X 中断方式向 APCI ID 为 0 的 CPU 发送中断所使用的地址，有关 x86 处理器 MSI-X 中断方式的详细说明见第 10.3 节。

在 x86 处理器中，Local APIC 寄存器空间是可变的，其基地址保存在 IA32_APIC_BASE 寄存器中，该寄存器是 x86 处理器的 MSR（Model Specific Register）寄存器，x86 处理器使用 RDMSR 和 WRMSR 指令访问这些寄存器。

PCI Express 配置空间占用 0xE000-0000~0xEFFF-FFFF 这段地址空间，下节将详细解释这段空间的使用方法。DMI Interface 负向译码空间被分为若干段，用来映射 ICH 使用的 PCI 总线地址空间。在 ICH 中提供了许多 PCI 设备，包括内嵌在 ICH 中的虚拟 PCI 设备和 PCIe 总线端口。这些 PCI 设备的 BAR 空间被映射到这段空间。

Montevina 平台使用 DMI 连接 MCH 和 ICH。当 CPU 对 PCI 空间发起数据请求时，这些数据首先到达 MCH，当 MCH 中的所有设备都不响应这个数据请求时，MCH 中的 DMI 接口设备将使用负向译码方式被动地接收这个数据请求，并将其转发到 ICH 中的 DMI 接口设备，从而到达 ICH。因此 Montevina 平台将这段空间称为“负向译码空间”。由以上说明可以发现与 MCH 连接的 PCIe 设备的访问延时小于与 ICH 连接中的 PCIe 设备。

5.3.2 PCIe 总线的配置空间

x86 处理器使用了两种机制访问 PCIe 设备的扩展配置空间。首先 x86 处理器提供了两个 I/O 端口寄存器，CONFIG_ADDRESS 和 CONFIG_DATA 寄存器，使用这两个寄存器访问 EP 配置空间的方法与访问 PCI 设备类似，详见第 2.2.4 节。然而这两个寄存器只能访问 PCI 设备的基本配置空间，即 PCIe 设备配置空间的前 256 个字节，而之后的扩展配置空间

需要通过 ECAM 方式进行访问。

在 Montevina 平台中，PCIe 设备配置空间的基地址保存在 PCIEXBAR 寄存器中，这段地址空间的大小为 256 MB，且为 256 MB 对齐。当 CPU 对 PCIe 设备配置空间^①进行读写访问时，MCH 将这个存储器读写请求转换为 PCI 配置读写总线周期后，再发送到相应的 PCIe 设备中。PCIe 总线规范将这种“对 PCIe 设备配置空间”的读写访问方式称为 ECAM 机制。

ECAM 机制的主要原理是，将处理器系统中所有 PCIe 设备的配置空间映射到一段地址连续的存储器域的地址空间中。CPU 可以直接对这段特殊的存储器域地址空间进行访问，从而访问 PCIe 设备的配置空间。

使用 ECAM 机制与使用 CONFIG_ADDRESS 和 CONFIG_DATA 这对寄存器，间接访问 PCIe 设备的配置空间有较大的不同。ECAM 机制是一种直接寻址方式，在 x86 处理器系统中，只有使用 ECAM 方式才可以访问 PCIe 设备的扩展配置空间。而其他处理器，如 PowerPC 处理器，即便不使用 ECAM 方式，也可以访问 PCIe 设备的扩展配置空间，在 MPC8548 处理器的 PEX_CONFIG_ADDR 寄存器^②中，设置了 EXT_REGN 字段（由 4 位组成），该字段可以与 REGN 字段组成一个 10 位的字段，从而可以访问所有扩展的 PCIe 设备配置空间。

Linux 系统定义了 raw_pci_read 和 raw_pci_write 两个函数，对 PCI 设备配置空间进行读写，这两个函数的实现在 ./arch/x86/pci/common.c 文件中，如源代码 5-1 所示。

源代码 5-1 raw_pci_read 和 raw_pci_write 函数

```
int raw_pci_read(unsigned int domain, unsigned int bus, unsigned int devfn,
                int reg, int len, u32 *val)
{
    if (domain == 0 && reg < 256 && raw_pci_ops)
        return raw_pci_ops->read(domain, bus, devfn, reg, len, val);
    if (raw_pci_ext_ops)
        return raw_pci_ext_ops->read(domain, bus, devfn, reg, len, val);
    return -EINVAL;
}

int raw_pci_write(unsigned int domain, unsigned int bus, unsigned int devfn,
                 int reg, int len, u32 val)
{
    if (domain == 0 && reg < 256 && raw_pci_ops)
        return raw_pci_ops->write(domain, bus, devfn, reg, len, val);
    if (raw_pci_ext_ops)
        return raw_pci_ext_ops->write(domain, bus, devfn, reg, len, val);
    return -EINVAL;
}
```

① 这段空间属于存储器域地址空间。

② 该寄存器与 PCI_CONFIG_ADDR 寄存器类似，用来访问 PCIe 设备的配置空间。

由以上代码，可以发现当 `raw_pci_read/write` 函数访问的配置寄存器号大于 256B 时，该函数将调用 `raw_pci_ext_ops` 函数，否则调用 `raw_pci_ops` 函数。其中 `raw_pci_ops` 函数指针使用 0xCF8 和 0xCFC 两个 I/O 端口寄存器访问 PCI 总线配置空间，而 `raw_pci_ext_ops` 函数使用 ECAM 方式访问 PCI 总线配置空间。

在 Linux x86 系统中，`raw_pci_ext_ops` 函数相当于 `pci_mmconfg` 函数，而 `pci_mmconfg` 函数指针分别指向两个函数，为 `pci_mmconfg_read` 和 `pci_mmconfg_write` 函数。这两个函数使用直接寻址方式（即 ECAM 方式）访问当前处理器系统中所有 PCIe 设备的扩展配置空间，其函数原型在 `./arch/x86/pci/mmconfig_32.c` 文件中。

Montevina 平台使用 256 MB 物理空间映射 PCI 设备的配置寄存器，因为在 Montevina 平台中有一棵 PCI 总线树，因此最多具有 256 条 PCI 总线，每一条 PCI 总线最多可以挂接 32 个 PCI 设备，每一个设备最多有 8 个 Function，而在每一个 Function 中最大的 PCI 总线配置寄存器空间为 4 KB。

因此将一棵 PCI 总线树上所有 PCI 设备的配置空间采用一一映射的方式对应到存储器域空间时，共需要 1 MB 空间，而一个 HOST 主桥可以管理的 PCI 总线为 256 条。为此 Montevina 平台共提供了 256 MB 大小的空间，其基地址在 PCIEXBAR 寄存器中，这段存储器域的地址空间与 PCI 总线的配置寄存器的对应关系如表 5-3 所示。

表 5-3 PCIEXBAR 空间与 PCI 总线配置寄存器的对应关系

地 址 偏 移	PCI 总线的配置空间
A[32:28]	与 PCIEXBAR 寄存器一致
A[27:20]	Bus Number
A[19:15]	Device Number
A[14:12]	Function Number
A[11:8]	Extended Register Number
A[7:2]	Register Number
A[1:0]	用作字节使能

当 CPU 对 PCIEXBAR 地址空间进行访问时，MCH 或者 ICH 将根据上表所示的规则，将这次存储器访问转化对 PCI 总线的某个设备进行的配置读写访问。如果当 CPU 对 PCIEXBAR+0x0811-0000 这个地址进行访问时，MCH 将这次地址访问转换为对 Bus 号为 0x81（A[27:20]为 0x81），Device 号为 1（A[19:15]为 1），且 Function 号为 0 的 PCI 设备配置空间的访问，访问的寄存器为 0x0（A[11:2]为 0）。

虽然采用 ECAM 方式可以访问之前使用 CONFIG_DATA 和 CONFIG_ADDRESS 寄存器不能访问的 PCI 扩展配置寄存器空间，但是也带来了一些问题。在一个处理器系统中，同一条 PCI 总线上的 Device Number 不一定连续，而且在多数 PCI 设备中也不会有 8 个 Function，这将造成 PCIEXBAR 空间会留有许多空洞。虽然 Montevina 平台提供了 256 MB 的 PCIe 设备使用的配置空间，但是这些空间的实际利用率较低。

PowerPC 处理器也可以使用 ECAM 方式映射 PCI 配置空间，如 MPC8548 处理器可以使用 PCIe 桥中的 Outbound 寄存器（PEXOWARn）将 PCI 配置空间映射到存储器域。此外 PowerPC 处理器还可以使用 PEX_CFG_DATA 和 PEX_CFG_ADDR 这两个寄存器访问扩展

PCIe 设备的配置空间。其中 PEX_CONFIG_ADDR 寄存器的结构如图 5-5 所示。



图 5-5 PEX_CONFIG_ADDR 寄存器的结构

PEX_CONFIG_ADDR 寄存器保存当前 PCIe 设备在处理器系统中的 ID 号，该寄存器的各个字段的描述如下。

- Enable 位。该位为 1 表示使能 HOST 处理器对 PCI 配置空间的访问，当 HOST 处理器对 PEX_CONFIG_DATA 寄存器进行访问时，HOST 主桥将对这个寄存器的访问转换为 PCI 配置读写周期并发送到 PCI 总线上。
- BUSN 字段记录 PCI 设备所在的总线号。
- DEVN 字段记录 PCI 设备的设备号。
- FUNCN 字段记录 PCI 设备的功能号。
- EXTREGN 和 REGN 字段，共 10 位，记录 PCI 设备的配置寄存器号。PowerPC 处理器使用这两个字段组成 10 位地址空间，从而可以访问 PCIe 设备使用的全部配置空间。

从原理上讲，x86 处理器也可以使用这种方式访问扩展的 PCI 配置空间，但是 x86 处理器没有采用这种方式，而是使用 ECAM 机制访问扩展的配置空间。这种方式将不可避免地在存储器域占用一段专用的地址空间。而这段地址空间的使用效率较低，因为在一条 PCI 总线上，PCIe 设备不会使用所有的设备号，而且每一个 PCIe 设备也不会使用所有的 Function 号，因此在这段地址空间中，有许多空间没有被充分利用。

5.4 小结

本章较为详细地介绍了 Montevina 平台与 PCIe 总线相关的内容。理解 PCIe 总线必须建立在理解处理器系统的基础之上，而 Intel 提供的处理器平台无疑是学习 PCIe 总线最合适的平台。Intel 是 PCIe 总线的缔造者，而且总是率先支持 PCIe 规范提出的最新功能。希望读者能够在充分理解处理器平台的基础上，进一步理解 PCIe 总线的细节知识。

本章因为篇幅有限，并不能对 Intel 的处理器平台进行详细分析与介绍。对这部分内容有兴趣的读者，可以首先阅读 Intel 提供的 Intel 64 and IA32 Architectures Software Developer's Manual 丛书（可以在 <http://www.intel.com/products/processor/manuals> 中下载）。

读者在获得这些入门知识之后，可以进一步阅读与 Intel 处理器相关的其他知识。目前 Intel 并没有完全公开其处理器平台的详细资料。但是从已有的资料中，也可以看到 Intel 在处理器领域的成就。目前 Intel 在这个领域处于无可争议的领袖地位。Intel 的处理器平台因为向前兼容的缘故，有些部件的实现并不完美，这些不完美是历史原因造成的。